Universidad EAFIT

Escuela de Ciencias Aplicadas e Ingeniería

Sistemas Operativos

Seguridad de archivos mediante intervención en la capa del kernel y mapeo de memoria (mmap)

Integrantes

Santiago Betancur

Daniel Correa Botero

Daniel Arango Guarín

Contenido

1. Introducción
2. Parte I. Análisis conceptual
3. Parte II. Estrategia con apoyo de inteligencia artificial
4. Parte III. Implementación y argumentación
5. Conclusiones

1. Introducción

La llamada mmap() proyecta archivos o dispositivos en el espacio de direcciones de un proceso. Una vez instalada la entrada de la tabla de páginas (PTE), las lecturas y escrituras se resuelven en la MMU sin pasar por el kernel, de modo que ningún mecanismo de espacio de usuario puede inspeccionarlas. Este trabajo diseña, implementa y evalúa una defensa ubicada en la capa del kernel que aporta control de acceso, aislamiento, confidencialidad, integridad y auditoría sobre regiones mapeadas.

La solución tiene dos componentes complementarios. El primero es un módulo de kernel, secure_vault, que expone el dispositivo /dev/secure_vault y aplica la política en el alta de la VMA, en el manejador de fallos de página y en la primera escritura de cada página. El segundo es un auditor global, kmod_auditor, que observa mediante un kprobe todas las llamadas mmap() del sistema en el hook security_mmap_file.

Los módulos se compilaron sin advertencias contra los headers de Linux 6.19.10 y 7.2.13, y se ejecutaron en una máquina virtual QEMU/KVM con el kernel 6.19.10 de Fedora 44. La batería de pruebas obtuvo 72 de 72 comprobaciones satisfactorias.

2. Parte I. Análisis conceptual

2.1 Mapeo virtual y fallos de página

La llamada mmap() entra por ksys_mmap_pgoff() y vm_mmap_pgoff(). Esta última invoca primero security_mmap_file(), punto de enganche de los módulos de seguridad (hook mmap_file). Luego toma el mmap_lock en escritura y ejecuta do_mmap(), que valida longitud y alineación, busca un hueco libre y calcula vm_flags a partir de prot y flags, añadiendo VM_MAYREAD, VM_MAYWRITE y VM_MAYEXEC. Para MAP_SHARED exige FMODE_WRITE si se pide escritura; para MAP_PRIVATE solo exige FMODE_READ.

mmap_region() crea el struct vm_area_struct con vm_start, vm_end, vm_pgoff, vm_flags y vm_file, invoca file->f_op->mmap(), inserta la VMA en el maple tree del proceso (mm->mm_mt) y calcula vm_page_prot. La llamada retorna sin haber creado ninguna PTE.

En el primer acceso, la MMU recorre CR3 → PGD → P4D → PUD → PMD → PTE y encuentra la entrada ausente. La CPU genera la excepción #PF (vector 14), apila el código de error y deja la dirección en CR2. El kernel atiende el fallo en exc_page_fault() y do_user_addr_fault(), bloquea la VMA (por RCU o con mmap_lock en lectura), comprueba que el acceso sea compatible con vm_flags y llama a handle_mm_fault(). Este reserva los niveles de tabla que falten y, en do_fault(), distingue lectura (do_read_fault), escritura privada (do_cow_fault) y escritura compartida (do_shared_fault). La página se obtiene del vm_ops->fault de la VMA, finish_fault() instala la PTE bajo el page-table lock y la CPU reejecuta la instrucción. Los accesos posteriores ocurren sin intervención del kernel; en las mediciones cuestan 0,5 ns.

2.2 MAP_SHARED frente a MAP_PRIVATE

La diferencia se representa en vma->vm_flags con VM_MAYSHARE (el usuario solicitó MAP_SHARED) y VM_SHARED (el mapeo es compartido y escribible). Un mapeo es de tipo copy-on-write cuando (vm_flags & (VM_SHARED | VM_MAYWRITE)) == VM_MAYWRITE (is_cow_mapping()).

En un mapeo compartido, las páginas son las del page cache del inodo (o las que entrega el driver). Las escrituras marcan el folio como sucio, son visibles de inmediato para cualquier otro proceso que mapee o lea el archivo y se persisten en disco. En un mapeo privado, la página del page cache se mapea en solo lectura y la primera escritura produce una copia anónima propia del proceso, que nunca llega al archivo.

Las implicaciones de seguridad son opuestas. El mapeo compartido permite la observación y modificación cruzada entre procesos, con efectos persistentes. El mapeo privado aísla las escrituras, pero rompe la suposición de inmutabilidad (pregunta 8) y es el escenario de Dirty COW (pregunta 3).

Cuando se hace un MAP_SHARED sobre un descriptor sin FMODE_WRITE, el kernel elimina VM_SHARED y conserva VM_MAYSHARE. Por ello, la política correcta para exigir mapeos compartidos se basa en VM_MAYSHARE. Una variante del módulo que usa VM_SHARED rechaza mapeos legítimos de solo lectura y hace fallar seis pruebas.

2.3 Ciclo de vida copy-on-write

En un mapeo privado, la PTE inicial apunta a la página compartida sin permiso de escritura, aunque la VMA tenga VM_WRITE. La escritura provoca un fallo de protección que atiende do_wp_page(). Si la página es anónima y exclusiva (PageAnonExclusive) se reutiliza activando escritura; en caso contrario wp_page_copy() asigna una página nueva, copia el contenido e instala una PTE escribible. Si la página proviene del page cache, do_cow_fault() crea directamente la copia anónima.

Las banderas relevantes son VM_MAYWRITE (permite activar escritura posteriormente), VM_WRITE (escritura actual), VM_SHARED/VM_MAYSHARE y, para accesos del kernel en nombre

de otro proceso, FOLL_WRITE | FOLL_FORCE, que permiten escribir en mapeos sin VM_WRITE siempre que exista VM_MAYWRITE.

En Dirty COW (CVE-2016-5195), un hilo escribía mediante /proc/self/mem (get_user_pages con FOLL_FORCE | FOLL_WRITE) en un mapeo privado de solo lectura de un archivo, mientras otro ejecutaba madvise(MADV_DONTNEED) en bucle. Tras provocar la copia COW, el código eliminaba FOLL_WRITE para reintentar sin exigir una PTE escribible. Si MADV_DONTNEED descartaba la copia en ese intervalo, el reintento obtenía la página original del page cache y la escritura terminaba sobre un archivo de solo lectura, como /etc/passwd o un binario setuid. La corrección introdujo FOLL_COW; en los kernels actuales esa bandera ya no existe y el modelo se apoya en PageAnonExclusive y en FAULT_FLAG_UNSHARE.

La solución implementada elimina este vector en su recurso: rechaza MAP_PRIVATE y, en mapeos de solo lectura, elimina VM_MAYWRITE, por lo que una escritura forzada mediante /proc/self/mem falla.

2.4 Intercepción: tabla de llamadas al sistema frente a LSM

Modificar la tabla de llamadas al sistema desde un módulo no es viable en kernels modernos. sys_call_table y kallsyms_lookup_name no están exportados a módulos: un módulo que los referencia no enlaza (modpost reporta “undefined”). La tabla reside en memoria de solo lectura y la técnica de desactivar CR0.WP ya no funciona, porque el kernel fija ese bit, lo restaura y emite una advertencia. La técnica además es frágil frente a CET, a la coexistencia de varias tablas (compat de 32 bits) y a los envoltorios __x64_sys_*, e introduce carreras entre CPU.

Un LSM es el mecanismo diseñado para este fin. Todo mmap() pasa por security_mmap_file() (hook mmap_file, nombre actual del antiguo file_mmap), que recibe el struct file, las credenciales y las protecciones solicitadas, y puede vetar la operación. Su limitación es que no puede cargarse como módulo: debe compilarse en el kernel o implementarse como programa BPF-LSM. La configuración de Fedora 44 incluye CONFIG_BPF_LSM y bpf en la lista de LSM activos, por lo que esta segunda vía es viable sin recompilar.

2.5 Cifrado transparente en memoria

En los sistemas de cifrado del kernel (fscrypt, eCryptfs, dm-crypt), el disco almacena datos cifrados y el sistema de archivos los descifra al cargarlos en el page cache del archivo lógico; mmap() proyecta esas páginas ya en claro. El aislamiento frente a procesos no autorizados no proviene de un page cache separado por sesión, sino de tres elementos: sin la clave cargada el archivo no puede abrirse, los permisos POSIX y las políticas LSM controlan quién lo abre, y la clave se gestiona en el keyring del kernel.

El módulo implementado aplica un modelo más estricto para su recurso. El respaldo en memoria contiene únicamente texto cifrado con AES-128-CTR, y cada apertura autorizada recibe una copia en claro privada que se materializa en el manejador de fallos y se borra al cerrar. Complementos necesarios son VM_DONTDUMP (excluir la región de los volcados de memoria), mlock o memfd_secret (retirar páginas del mapa directo) y el cifrado de memoria por hardware (SME, SEV, TDX) frente a lecturas físicas.

2.6 Sincronización e integridad: msync y fsync

En un mapeo compartido escribible, el kernel instala inicialmente las PTE en solo lectura para invocar vm_ops->page_mkwrite en la primera escritura de cada página; el folio se marca sucio y la PTE recibe el bit D. El proceso de writeback persiste las páginas sucias y, antes de hacerlo, limpia el bit D y vuelve a proteger la PTE (folio_mkclean), de modo que la siguiente escritura vuelve a notificarse. msync(MS_SYNC) fuerza y espera la persistencia del rango; fsync() persiste datos y metadatos del archivo.

Una escritura a través de un mapeo es una instrucción de almacenamiento que no genera la llamada write(). Por ello, los filtros que inspeccionan datos en vfs_write o en los hooks de escritura del LSM no la observan. Los únicos puntos de observación son page_mkwrite, el writeback y msync/fsync. El módulo usa page_mkwrite para auditar la primera escritura de cada página y marcarla como modificada; las páginas modificadas se vuelven a cifrar al cerrar el mapeo. Se verificó que page_mkwrite se invoca en este tipo de mapeo, y que para páginas sin address_space debe devolver la página bloqueada con VM_FAULT_LOCKED.

2.7 Control de accesos con eBPF

eBPF permite instrumentar el kernel sin recompilarlo mediante kprobes, fentry/fexit (con tipos obtenidos de BTF) o tracepoints como syscalls:sys_enter_mmap. Un programa enganchado en security_mmap_file o vm_mmap_pgoff obtiene prot, flags, la ruta del archivo, el PID y las credenciales. Puede enviar eventos a espacio de usuario con un ring buffer (BPF_MAP_TYPE_RINGBUF) para detectar patrones anómalos: memoria simultáneamente escribible y ejecutable, escrituras compartidas sobre rutas sensibles o ráfagas de mapeos. El verificador garantiza la terminación y los accesos acotados del programa.

Los kprobes y fexit observan pero no pueden vetar; para bloquear se requiere BPF-LSM (lsm/mmap_file). En este trabajo el auditor se implementó como módulo con register_kprobe() sobre security_mmap_file, el mismo punto de enganche que usaría un programa eBPF. En la prueba registró 32 mapeos, marcó 6 como anómalos y no perdió eventos.

2.8 Evasión de permisos POSIX

Un proceso que abre un archivo con O_RDONLY puede mapearlo con PROT_READ | PROT_WRITE y MAP_PRIVATE. do_mmap() solo exige FMODE_READ para mapeos privados, porque las escrituras se dirigen a copias anónimas y nunca al archivo. El experimento de control lo confirma: el mapeo se concede, la escritura funciona y el archivo permanece intacto. La misma combinación con MAP_SHARED se rechaza con EACCES.

Los riesgos son la alteración temporal engañosa (el proceso trabaja con datos modificados mientras los permisos sugieren inmutabilidad), la fuga de información derivada si la región se comparte con otro componente y la dependencia de la corrección del mecanismo COW. El módulo rechaza cualquier MAP_PRIVATE sobre su recurso.

2.9 Mitigación de carreras con madvise(MADV_DONTNEED)

madvise(MADV_DONTNEED) toma el mmap_lock en lectura (o el bloqueo por VMA) y elimina PTE bajo el page-table lock, sin modificar datos. La protección actual combina tres mecanismos. Primero, la revalidación bajo el page-table lock: los manejadores conservan la PTE original y comprueban con pte_same() que no haya cambiado antes de instalarla o promoverla; si cambió, reintentan. Segundo, la intención de escritura ya no se descarta: get_user_pages exige una página anónima exclusiva y escribible, y si la copia se perdió vuelve a fallar con intención de escritura. Tercero, la página se fija (FOLL_PIN) después de verificar la PTE, de modo que una eliminación posterior no cambia la página que se escribe.

La prueba de carrera reproduce el patrón de Dirty COW contra el módulo propio: un hilo intenta escrituras forzadas mediante /proc/self/mem mientras otro ejecuta MADV_DONTNEED. Se registraron 717 778 intentos sin ningún éxito, el contenido quedó intacto y la verificación de integridad no detectó páginas alteradas.

2.10 Aislamiento de memoria y paginación

struct page (o struct folio) describe cada marco físico con sus banderas, su contador de referencias y su contador de mapeos. Un manejador de fallos incrementa la referencia con get_page() para que el marco no se libere mientras una PTE lo use. Las tablas multinivel PGD, P4D, PUD, PMD y PTE, cuya raíz indica CR3, dan a cada proceso un espacio de direcciones independiente.

En x86-64, la PTE contiene los bits P (presente), R/W (escritura), U/S (usuario o supervisor), D (sucia) y NX (no ejecutable, bit 63). NX impide ejecutar datos y es la base de la política W^X. R/W implementa el COW y la notificación de primera escritura. U/S impide que el código de usuario acceda a memoria del kernel, mientras SMEP y SMAP impiden que el kernel ejecute o lea memoria de usuario fuera de copy_from_user y copy_to_user.

vm_get_page_prot() traduce vm_flags a bits de protección, y mprotect() solo puede activar permisos cubiertos por las banderas VM_MAY*. Por eso el módulo rechaza PROT_EXEC y además elimina VM_MAYEXEC, lo que hace imposible convertir la región en ejecutable incluso con un mprotect() posterior.

3. Parte II. Estrategia con apoyo de inteligencia artificial

3.1 Flujo de trabajo

La inteligencia artificial participó en tres fases. En la exploración arquitectónica se usó para relacionar el código del repositorio base (pasos 4 y 6) con las estructuras del kernel (vm_area_struct, vm_operations_struct, hooks LSM) y para comparar alternativas de intercepción. En la generación de código produjo los borradores del módulo, de la capa criptográfica, del auditor y de las pruebas. En la validación diseñó casos de ataque controlados y la infraestructura de pruebas.

Ninguna salida se aceptó sin verificación. Cada símbolo se contrastó con los headers del kernel instalado, el código se compiló con advertencias tratadas como errores contra dos versiones del kernel y se ejecutó en una máquina virtual desechable. Además, se aplicó una

prueba de mutación para comprobar que las pruebas detectan la eliminación de cada control.

3.2 Prompts utilizados

Los siguientes prompts guiaron la construcción del proyecto. Sus respuestas se contrastaron con un segundo modelo y coincidieron con la implementación final; las diferencias encontradas en las respuestas iniciales se registran en la sección 3.3.

Prompt 1. Exploración arquitectónica

Rol: ingeniero del subsistema de memoria de Linux.

Contexto: driver de caracteres que expone un buffer con `.mmap`, mediante `remap_pfn_range` y `vm_ops->fault` (código de los pasos 4 y 6), en Linux 6.19.

Instrucción y restricciones: explicar el flujo exacto desde `mmap()` hasta el primer fallo de página con las funciones reales del kernel e identificar los puntos donde un módulo puede intervenir (`f_op->mmap`, `vm_ops->fault`, `page_mkwrite`, hook `mmap_file`); citar el header de cada símbolo y señalar lo que no pueda verificarse; no inventar nombres.

Resultado esperado: secuencia de eventos y tabla de puntos de intercepción con su header.

Resultado obtenido: la secuencia coincidió con la descrita en la pregunta 1. La verificación en headers corrigió la referencia a `mm->mm_rb`, sustituida por el maple tree desde Linux 6.1.

Prompt 2. Decisión de arquitectura

Rol: arquitecto de seguridad de sistemas.

Contexto: reforzar la seguridad de regiones `mmap` desde el kernel, sin recompilarlo, sin privilegios de administrador en el equipo de desarrollo y con QEMU/KVM disponible.

Instrucción y restricciones: comparar el hook de la tabla de llamadas, un LSM compilado, BPF-LSM, kprobes/fexit y un dispositivo propio con `.mmap` y `.fault` según viabilidad en 6.x, cobertura, capacidad de bloqueo, riesgo de fallo del kernel y sobrecosto; proponer un experimento mínimo para cada afirmación sobre exportación de símbolos o protección de `CR0.WP`.

Resultado esperado: matriz de compensaciones, recomendación justificada y experimentos de verificación.

Resultado obtenido: se adoptó la combinación de dispositivo propio y auditor por kprobe. Los experimentos confirmaron que `sys_call_table` y `kallsyms_lookup_name` no enlazan y que `CR0.WP` no puede desactivarse.

Prompt 3. Módulo y política de acceso

Rol: desarrollador de drivers de Linux.

Contexto: módulo /dev/secure_vault de cuatro páginas basado en el patrón vm_ops->fault.

Instrucción y restricciones: implementar open, mmap, fault, close y release con control por UID o CAP_SYS_ADMIN (verificado de nuevo en mmap), rechazo de PROT_EXEC y limpieza de VM_MAYEXEC, exclusividad de MAP_SHARED, validación de vm_pgoff y longitud sin desbordamiento, y banderas VM_DONTCOPY, VM_DONTDUMP y VM_DONTEXPAND; compilar con -Wall -Wextra -Werror en 6.x y 7.x usando vm_flags_set/clear y class_create con guarda de versión; explicar qué bandera identifica MAP_SHARED en un descriptor de solo lectura.

Resultado esperado: código modular y lista de errores devueltos por cada control.

Resultado obtenido: la estructura coincidió con kmod_secure_mmap.c y secure_policy.h. La bandera correcta es VM_MAYSHARE, lo que corrigió la indicación inicial del plan de exigir VM_SHARED.

Prompt 4. Cifrado en reposo e integridad

Rol: especialista en criptografía aplicada en el kernel.

Contexto: el respaldo de la bóveda debe permanecer cifrado en memoria y cada apertura recibe una copia en claro privada que se descifra en el fallo de página.

Instrucción y restricciones: implementar un envoltorio sobre crypto_alloc_skcipher("ctr(aes)") y crypto_alloc_shash("hmac(sha256)"); justificar el vector de inicialización para no reutilizar el flujo de clave al volver a cifrar; comprobar todos los retornos; usar memzero_explicit, kfree_sensitive y comparación en tiempo constante; indicar el header de cada función en 6.19.

Resultado esperado: secure_crypto.c y explicación de por qué el modo CTR sin MAC no detecta manipulación.

Resultado obtenido: coincidió con la implementación: vector de inicialización formado por índice de página y generación, y HMAC-SHA256 por página. La verificación mostró que crypto_memneq cambió de header entre versiones, lo que se resolvió con __has_include.

Prompt 5. Detección de escrituras y persistencia

Rol: especialista en la ruta de fallos de escritura del subsistema de memoria.

Contexto: mapeo MAP_SHARED y escribible sobre páginas propias del driver, sin address_space.

Instrucción y restricciones: implementar vm_ops->page_mkwrite para auditar la primera escritura y marcar la página como modificada, y un proceso de confirmación al cerrar que vuelva a cifrar solo las páginas modificadas; explicar cuándo el kernel instala la PTE en solo lectura, qué debe devolver page_mkwrite para evitar reintentos indefinidos y qué orden de bloqueos evita interbloqueos.

Resultado esperado: código y prueba que demuestre empíricamente la invocación de page_mkwrite.

Resultado obtenido: coincidió con la implementación (VM_FAULT_LOCKED, orden de bloqueo sesión antes que respaldo). La prueba confirmó la invocación y la optimización redujo el costo de munmap de 11 µs a 2 µs.

Prompt 6. Pruebas de ataque controladas

Rol: auditor de seguridad ofensiva en entorno de laboratorio.

Contexto: el único objetivo es /dev/secure_vault dentro de una máquina virtual desechable.

Instrucción y restricciones: escribir programas en C que informen el resultado de cada comprobación para UID no autorizado, descriptor heredado tras fork y setuid, PROT_EXEC y mprotect(PROT_EXEC), MAP_PRIVATE (con el experimento de control de la pregunta 8), desplazamiento fuera de rango, escritura forzada mediante /proc/self/mem, carrera con madvise(MADV_DONTNEED) y manipulación de un bit; añadir una matriz que ejecute los mismos ataques contra el paso 4 sin endurecer; sin cargas de escalada de privilegios ni acceso a archivos del sistema.

Resultado esperado: carpeta de pruebas con manifiesto de UID y script de ejecución.

Resultado obtenido: coincidió con tests/ (diez programas, 72 comprobaciones) y attack_matrix.c. La revisión corrigió una prueba que esperaba SIGBUS en un proceso hijo que, por VM_DONTCOPY, no hereda el mapeo.

Prompt 7. Infraestructura de pruebas en máquina virtual

Rol: ingeniero de integración continua para el kernel.

Contexto: sin privilegios de administrador, sin kernel-devel instalado y con una ruta del repositorio que contiene espacios; disponibles qemu-system-x86_64, /dev/kvm y las imágenes del kernel en /boot.

Instrucción y restricciones: crear un script que obtenga los headers sin privilegios, compile los módulos, construya un initramfs con un proceso init en C que ejecute un plan de pruebas (cargar módulos, ejecutar con un UID dado, leer el registro del kernel, descargar), arranque la máquina virtual y guarde los registros como evidencia; no cargar nada en el equipo anfitrión; resolver la limitación de kbuild con rutas con espacios; incluir una prueba de mutación.

Resultado esperado: carpeta vm/ con el lanzador, el init y la prueba de mutación.

Resultado obtenido: coincidió con vm/run_vm_tests.sh, vm/init.c y vm/mutation_test.sh. La prueba de mutación detectó siete de ocho mutantes.

Prompt 8. Medición de sobrecosto y verificación de afirmaciones

Rol: ingeniero de rendimiento y revisor crítico.

Contexto: comparación entre el paso 4 (remap_pfn_range) y la bóveda.

Instrucción y restricciones: escribir un benchmark de latencia de mmap, costo por fallo de página, munmap, sesión completa y acceso estacionario, con 30 corridas, media, desviación estándar y fallos menores; un script con perf stat, strace -c y ftrace function_graph sobre vault_fault; y módulos mínimos que pongan a prueba afirmaciones frecuentes (sys_call_table, kallsyms_lookup_name, escritura directa de vm_flags, class_create con dos argumentos, kmap_atomic, desactivación de CR0.WP); repetir en varios arranques, advertir que function_graph infla las duraciones y no afirmar la existencia de una API sin experimento.

Resultado esperado: benchmark, verificación de afirmaciones y tabla de sobrecosto interpretada.

Resultado obtenido: coincidió con bench/ y verificacion_ia/. Se comprobó que kmap_atomic aún compila (está obsoleta, no eliminada), contrariamente a lo afirmado en el borrador inicial.

3.3 Alucinaciones y errores detectados

Afirmación o práctica sugerida	Verificación	Resultado
Usar `sys_call_table` desde un módulo	Compilación contra 6.19 y 7.2	No enlaza: símbolo no exportado
Localizar la tabla con `kallsyms_lookup_name`	Compilación contra 6.19 y 7.2	No enlaza: no exportado desde 5.7
Desactivar `CR0.WP` con `write_cr0`	Ejecución en la máquina virtual	El bit permanece activo (CR0 = 0x80050033) y el kernel emite una advertencia
Modificar `vma->vm_flags` directamente	Compilación	Error: miembro de solo lectura desde 6.3
`class_create(THIS_MODULE, nombre)`	Compilación	Error de tipos: firma de un argumento desde 6.4
`kmap_atomic` ya no existe	Compilación	Compila: está obsoleta, se prefiere `kmap_local_page`
Exigir `VM_SHARED` para admitir solo `MAP_SHARED`	Prueba de mutación	Rechaza mapeos legítimos de solo lectura; corresponde `VM_MAYSHARE`
El paso 4 permite mapear fuera de su buffer	Matriz de ataques	Ignora el desplazamiento, pero su límite de tamaño impide la lectura fuera del buffer
Limpiar `VM_MAYWRITE` impide `mprotect(PROT_WRITE)`	Prueba de mutación	Control redundante: el kernel ya lo hace en mapeos compartidos de solo lectura
`mm->mm_rb` organiza las VMA	Revisión de headers	Sustituido por el maple tree (`mm->mm_mt`)

3.4 Procedimiento de validación

1. Contraste de cada símbolo con los headers del kernel (kernel-devel de Fedora 44, versiones 6.19.10 y 7.2.13), obtenidos sin privilegios mediante dnf download y rpm2cpio.
2. Compilación de los seis módulos del repositorio contra ambas versiones con -Wall -Wextra -Werror: doce compilaciones sin advertencias.
3. Ejecución en una máquina virtual QEMU/KVM con el kernel 6.19.10. Ningún módulo se cargó en el equipo anfitrión.
4. Batería de diez programas de prueba con 72 comprobaciones, repetida tres veces con resultado idéntico.
5. Prueba de mutación: eliminación de cada control por separado, exigiendo que alguna prueba falle.
6. Revisión manual del orden de bloqueos, de la gestión de referencias (get_page, put_page), de las rutas de error y del borrado de material sensible.

3.5 Arquitectura seleccionada

La arquitectura combina profundidad y amplitud. El auditor kmod_auditor registra un kprobe en security_mmap_file, el punto que el kernel ejecuta antes de do_mmap() para todos los mapeos del sistema. Clasifica cada evento según reglas de anomalía y lo expone en /proc/mmap_audit.

El módulo secure_vault actúa sobre su propio recurso en cuatro momentos:

1. Apertura: verifica la identidad y crea una sesión privada.
2. Alta de la VMA (vault_mmap): aplica los controles de identidad, W^X, mapeo compartido, límites y aislamiento.
3. Fallo de página (vault_fault): descifra la página en la copia privada de la sesión y verifica su HMAC antes de entregarla.
4. Primera escritura (vault_page_mkwrite): la audita y marca la página como modificada. Al cerrar, vault_commit_session vuelve a cifrar las páginas modificadas con una nueva generación y actualiza su HMAC.

3.6 Matriz de compensaciones

Decisión	Seguridad	Rendimiento medido	Complejidad
Validación en `.mmap`	Alta sobre el recurso; no observa accesos posteriores	`mmap()` 0,41 veces el del paso 4	Baja
Descifrado y HMAC en el fallo de página	Confidencialidad e integridad por página	Sesión completa 2,14 veces la del paso 4	Alta
Auditoría por página y `page_mkwrite`	Visibilidad de cada página tocada y de la primera escritura	1,7 µs por fallo frente a 0,145 µs con PTE precargada	Media
Confirmación solo de páginas modificadas	Detección de cambios autorizados al cerrar	`munmap()` 1,08 veces el del paso 4	Media
Acceso con PTE instalada	Sin inspección adicional	Igual al paso 4 (0,5 ns)	Ninguna
Kprobe global	Cobertura de todo el sistema, sin capacidad de bloqueo	Un manejador por `mmap()`, sin eventos perdidos	Baja
Hook de la tabla de llamadas	Cobertura global teórica	No aplica	Inviable
BPF-LSM (trabajo futuro)	Cobertura global con bloqueo	Bajo	Media

4. Parte III. Implementación y argumentación

4.1 Componentes entregados

Componente	Descripción
`paso10_secure_mmap/`	Módulo `secure_vault` dividido en política (`secure_policy.h`), criptografía (`secure_crypto.c`), auditoría (`secure_audit.c`) e interfaz de usuario (`secure_vault_uapi.h`), con pruebas, benchmark y verificación de afirmaciones
`paso9_kprobe_auditor/`	Auditor global de `mmap()` mediante kprobe y programa generador de casos
`paso8_secure_driver/`	Componente previo de control de acceso con CRC32, corregido para compilar sin advertencias
`vm/`	Infraestructura de ejecución en máquina virtual, compatibilidad, mutación y benchmark repetido
`docs/evidencia/`	Registros de todas las ejecuciones

4.2 Controles de seguridad

Control	Mecanismo	Prueba
Control de acceso	UID autorizado o `CAP_SYS_ADMIN`, verificado en `open`, `mmap` e `ioctl`	`test_unauthorized`, `test_fd_inherit`
W^X	Rechazo de `VM_EXEC` y eliminación de `VM_MAYEXEC`	`test_prot_exec`
Solo lectura efectiva	Eliminación de `VM_WRITE` y `VM_MAYWRITE` si el descriptor no es escribible	`test_ro_write`
Solo mapeos compartidos	Rechazo cuando falta `VM_MAYSHARE`	`test_private_map`
Límites	Desplazamiento y longitud dentro de la bóveda	`test_oob_pgoff`
Aislamiento	`VM_DONTCOPY`, `VM_DONTDUMP`, `VM_DONTEXPAND`	`test_oob_pgoff`
Auditoría	Contadores atómicos, `/proc/secure_vault/stats` y registro del kernel con límite de tasa	Todas
Cifrado en reposo	AES-128-CTR con copia en claro privada por apertura	`test_crypto_rest`
Integridad	HMAC-SHA256 por página; `SIGBUS` ante manipulación	`test_tamper`, `test_race_madvise`
Detección de escrituras	`page_mkwrite` y registro de páginas modificadas	`test_authorized`

4.3 Resultados de las pruebas

Las diez pruebas ejecutadas en la máquina virtual superaron sus 72 comprobaciones. Un usuario no autorizado recibe EPERM al abrir el dispositivo. Un descriptor heredado por un proceso que cambió a un UID no autorizado no puede mapearse. Los mapeos ejecutables, privados o fuera de rango se rechazan. La escritura forzada sobre un mapeo de solo lectura falla. La alteración de un bit del respaldo cifrado se detecta y el acceso a la página afectada termina con SIGBUS, sin entregar datos corruptos.

La matriz de ataques compara el módulo con el driver del paso 4:

Ataque	Paso 4	secure_vault
Apertura por un usuario no autorizado	Permitido	Bloqueado (EPERM)
`mmap` con `PROT_EXEC`	Permitido	Bloqueado (EACCES)
`mprotect(PROT_EXEC)` posterior	Permitido	Bloqueado (EACCES)
`MAP_PRIVATE` con escritura	Permitido	Bloqueado (EACCES)
Desplazamiento ignorado (alias de la página 0)	Permitido	Bloqueado
Desplazamiento de 1 MiB fuera del dispositivo	Permitido	Bloqueado (EINVAL)
Longitud mayor que el buffer	Bloqueado	Bloqueado
Lectura desde un proceso hijo tras `fork`	Permitido	Bloqueado (SIGSEGV)
Escritura forzada en mapeo de solo lectura	Bloqueado	Bloqueado

La prueba de mutación eliminó cada control por separado: siete de ocho variantes fueron detectadas. La variante no detectada corresponde al control de solo lectura, redundante con el comportamiento del kernel, y se conserva como defensa en profundidad.

4.4 Justificación de la solución en el kernel

Tras el primer fallo de página, los accesos a la región no ejecutan código del kernel ni de bibliotecas: el acceso medido cuesta 0,5 ns tanto en el paso 4 como en la bóveda. Una biblioteca interpuesta con LD_PRELOAD solo observa la llamada mmap() inicial y puede evitarse con llamadas directas al sistema, binarios estáticos o ptrace.

Solo el kernel controla los elementos que determinan la seguridad: las banderas de la VMA, el manejador de fallos (donde ocurre el descifrado y la verificación), la notificación de primera escritura y las credenciales del proceso. La segunda verificación de identidad en mmap protege incluso frente a descriptores ya abiertos que cambian de manos, algo que ninguna capa de usuario puede garantizar. El texto en claro solo existe en la copia privada de cada apertura y se borra al cerrar.

4.5 Medición del sobrecosto

Las latencias se midieron con clock_gettime y getrusage en espacio de usuario, con ktime_get_ns() dentro del manejador de fallos y con ftrace function_graph. Cada arranque de la máquina virtual ejecutó 30 corridas de 2000 iteraciones. La tabla reporta la mediana de seis arranques válidos, porque el ruido del equipo anfitrión entre arranques llegó a duplicar los valores absolutos.

Métrica	Paso 4	secure_vault	Razón
`mmap()` de cuatro páginas	6835 ns	2836 ns	0,41
Primer acceso a cuatro páginas	580 ns	6717 ns	11,6
Costo por página en el primer acceso	145 ns	1679 ns	11,6
`munmap()`	3076 ns	3324 ns	1,08
Mapeo, primer acceso y desmapeo	10 491 ns	12 894 ns	1,23
Sesión completa con descriptor nuevo	15 535 ns	33 175 ns	2,14
Acceso con PTE instalada	0,5 ns	0,5 ns	1,00

El sobrecosto se concentra en eventos poco frecuentes (alta de la VMA, primer acceso a cada página y cierre) y es proporcional al número de páginas tocadas, no al número de accesos. El mmap() de la bóveda es más rápido porque no precarga PTE, mientras el paso 4 las instala con remap_pfn_range; en consecuencia, la bóveda registra cuatro fallos menores por mapeo y el paso 4 ninguno. El costo dominante en el fallo es el descifrado y la verificación criptográfica. El trazado con function_graph confirmó la ruta de ejecución (vault_decrypt_verify, crypto_skcipher_encrypt con la implementación AES de la CPU), aunque sus duraciones absolutas están infladas por la propia instrumentación. perf y strace no estaban disponibles en el entorno de pruebas; el repositorio incluye un script que los utiliza cuando están instalados.

5. Conclusiones

La seguridad de una región mapeada con `mmap()` solo puede garantizarse en el kernel, porque después del primer fallo de página los accesos no atraviesan ningún punto observable desde espacio de usuario. La solución implementada aplica controles en el alta de la VMA, en el fallo de página y en la primera escritura, y los complementa con un auditor global. Los resultados muestran que bloquea los ataques que el driver de referencia permite. El sobrecosto se limita a los eventos de mapeo, primer acceso y cierre, mientras los accesos posteriores conservan la velocidad del hardware.

El uso de inteligencia artificial aceleró la exploración y la escritura del primer borrador. Sin embargo, el valor del trabajo dependió de la verificación sistemática contra el kernel real.